Final Project:
Point-to-Point (P2P) Messaging

Computer Security 4173-001

Natalie Roman

113530166

May 07th, 2026

Introduction

For the final project, I designed a secure instant messaging tool for "Alice" and "Bob". With this tool, they are able to send messages to each other, but they must have the same password to access it and correctly encrypt and decrypt messages shared between one another. Per project instructions, each message during Internet transmission is encrypted using a key with length no less than 56 bits. For my tech stack I chose Python as I am more fluent in it, tkinter for the GUI, and for the cryptography implementations I cited from cryptography.io and the official Python documentation.

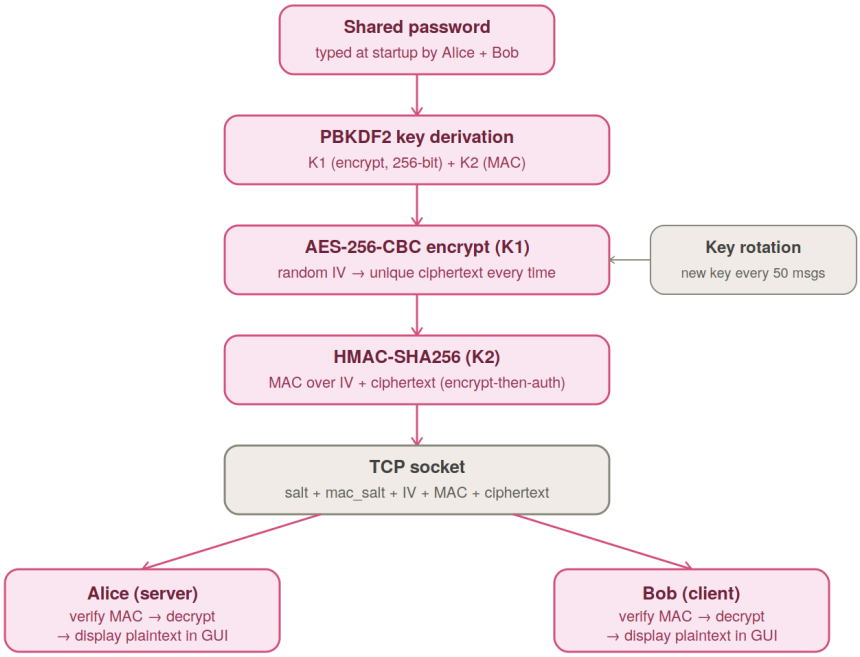
The design issues and requirements that I had to consider are listed below:

- With a key no less than 56 bits, what cipher should you use?
- DO NOT directly use the password as the key, how can you generate the same key between Alice and Bob to encrypt messages?
- What will be used for padding?
- A graphical user interface (GUI) is required. When sending a message, display the sent ciphertext. When receiving a message, display the received ciphertext and decrypted plaintext.
- How should Alice and Bob set up an initial connection and also maintain the connection with each other on the Internet?
- If Alice or Bob sends the same message multiple times, it is desirable to generate a different ciphertext each time. How to implement this?
- Design a key management mechanism to periodically update the key used between Alice and Bob. Justify why the design can enhance security

I submitted 5 files. ***encryptengine.py*** is where all the cryptography functions are at: key derivation, encryption, decryption, MAC, and key rotation are all in there. ***alice_gui.py*** and ***bob_gui.py*** are the actual messenger windows, Alice runs hers first and Bob connects to her. Each design decision is explained in the sections below with screenshots.

To run the GUI: uncompress the ZIP file, ‘cd’ into the project folder, the following commands are:
>> python alice_gui.py
(open another terminal tab,without closing alice’s)
>> python bob_gui.py
Both messenger windows should be open and ready to use.

System Design



The diagram above shows the full flow of the system from the first steps implemented to the end. Everything starts with a shared password that both Alice and Bob type in the beginning. Per project instructions, the password itself should never get used directly as a key, so I implemented a derive key function that runs it through PBKDF2 first which produces two keys:

K1: for encryption

K2: for the MAC

Every message then gets encrypted with AES-256-CBC using a random IV, authenticated with HMAC, and sent over a TCP socket.

Step 1: Key Derivation (*encryptengine.py*)

Requirement: "DO NOT directly use the password as the key, how can you generate the same key between Alice and Bob?"

The first thing I implemented within this project was the function derive_key to perform key derivation and I used PBKDF2HMAC from cryptography.io to stretch the password into a proper 256-bit key. In class we had learned that 128 bits is the current standard but 256 bits is still pretty common, they do both work with the project requirements having our key be no less than 56 bits. PBKDF2 runs SHA-256

internally 1,200,000 times which makes brute force attacks harder on this program because of all of the iterations, I mainly got that number because it was suggested by the site I cited. It was also suggested that a random 16-byte salt is generated so that even if two users have the same password, their keys are still different.

```python
def derive_key(password: str, salt: bytes = None):
    salt_length = 16
    iterations = 1_200_000
    key_length = 32

    if salt is None:
        # randomly generate salt if not given
        salt = os.urandom(salt_length)

    # derive
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=key_length,
        salt=salt,
        iterations=iterations,
    )

    key = kdf.derive(password.encode())
    return key, salt
```

Alice generates the salt and sends it to Bob. Bob uses the same salt with the same password and gets the same key. So they never send the actual key yet they are still matching and authenticating each other. Two keys are derived, K1 for encryption and K2 for the MAC (another requirement) which is why we end up calling derive_key twice.

```
(base) natalie@natalie-roman-2 CompSecurity_FinalProject % python encryptengine.
py
Alice's key: 1bb5c53729c5480da32ffa9e55d7484b37bb59a07c8277ce586a81bdd18dff94
Bob's key: 1bb5c53729c5480da32ffa9e55d7484b37bb59a07c8277ce586a81bdd18dff94
Keys match:  True
MAC keys match: True
```

Step 2: Encryption (*encryptengine.py*)

Requirement:"With a key no less than 56 bits, what cipher should you use?" and "What will be used for padding?" and "If Alice or Bob sends the same message multiple times, it is desirable to generate a different ciphertext each time."

I chose AES-256-CBC as the cipher because we had learned in class that AES is standard for modern attacks and also supports the key size of 128, 192 and 256. For padding I used PKCS7, we know that block ciphers require plaintext to be an exact multiple of the block size and AES uses 128-bit blocks so PKCS7 fills the last block to match the requirement and removes the padding on decryption. Once again, cryptography.io is beautiful and has implementations regarding this and regarding using a random IV to generate every message. This helps with having a different cipher text each time.

```python
def encrypt(key: bytes, plaintext: str):
    # the random iv solves the same message issue, make sure there is unque
    # cited from https://cryptography.io/en/latest/hazmat/primitives/padding
    iv = os.urandom(16) # AES block size

    global block_size
    block_size = 128 # AES block size in bits
    padder = padding.PKCS7(block_size).padder()
    padded_data = padder.update(plaintext.encode()) + padder.finalize()

    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(padded_data) + encryptor.finalize()

    return iv, ciphertext
```

Step 3: Message Authentication (*encryptengine.py*)

Requirement: To make sure the message wasn't modified!

In Lecture 8 we learned that encryption alone does not protect integrity of the message and that someone could flip bits in the cipher text and Bob would decrypt a modified message without knowing it is tampered. HMAC-SHA256 was implemented using the second separate key K2. Just like in lecture, using two separate keys K1 and K2 is important, one for encryption and the other for MAC. The MAC is computed over IV + ciphertext. Bob checks the MAC before he tries to decrypt. We know that Encrypt-then-Authenticate is the most secure combination, in that order so that is why I implemented this.

```python
def generate_mac(mac_key: bytes, ciphertext: bytes):
    h = hmac.HMAC(mac_key, hashes.SHA256())
    h.update(ciphertext)
    mac = h.finalize()
    return mac

def verify_mac(mac_key: bytes, ciphertext: bytes, mac: bytes):
    h = hmac.HMAC(mac_key, hashes.SHA256())
    h.update(ciphertext)
    try:
        h.verify(mac)
        return True
    except InvalidSignature:
        return False
```

Step 4: Key Rotation (*encryptengine.py*)

Requirement: "Design a key management mechanism to periodically update the key used between Alice and Bob. Justify why the design can enhance security."

For this requirement, I implemented a counter to track how many messages have been sent. After the 50 messages, a new salt will be generated and both sides will derive a new key from it. Alice will send the new salt to Bob through the socket so he can re-derive the same key without the actual key again.

```python
rotation = 50  # rotate every 50 messages
message_count = 0

def rotate_key(password: str, current_key: bytes, current_salt: bytes):
    global message_count
    message_count += 1
    print(f"Message count: {message_count}/{rotation}")

    if message_count >= rotation:
        message_count = 0

        new_key, new_salt = derive_key(password)
        print("Key rotated! New salt generated and new key derived")

        return new_key, new_salt
    return current_key, current_salt
```

We know that one-time pads are more secure but because that isnt as realistic, the approach of rotating the key every 50 messages will limit the key being exposed and can also lock an attacker out after 50 messages if they were to figure it out.

```
Message count: 2/50
Ciphertext 2: 5a3d93bce03834ec6fb5feb2b67e1c11536b042b9bc948381378db1481524975
Decrypted plaintext 2: I hope he gives me a 100%
Same message, different ciphertext? True
```

Step 5: Networking (*alice_gui.py/bob_gui.py*)

Requirement: "How should Alice and Bob set up an initial connection and also maintain the connection with each other on the Internet?"

Python has a built-in `socket` library and I used it with TCP.. I chose it because it guarantees packets arrive in order and without being corrupted. Alice runs her file first and binds to port 65432 as the server. Bob connects to that same address on the client side. Once both sides are connected, both sides run two threads, one for sending and one for receiving. So neither ends up waiting before new messages come in.

```python
def start_server():
    global conn
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    conn, addr = server.accept()
```

Alice starting on server side

```python
# connect to Alice
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((HOST, PORT))
```

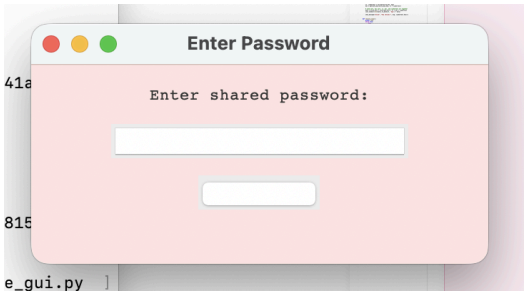
Bob connecting on client side

When a message is sent, all pieces are packed in one block -> salt + mac_salt + iv + mac + ciphertext. The receiver unpacks them in the same order. The length is sent as 4 bytes so the receivers knows how many bytes to read.

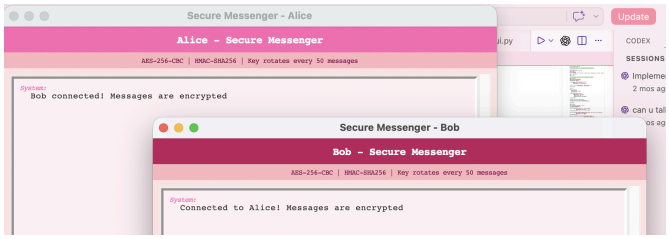
Step 6: GUI (*alice_gui.py/bob_gui.py*)

Requirement: "A graphical user interface (GUI) is required. When sending a message, display the sent ciphertext. When receiving a message, display the received ciphertext and decrypted plaintext."

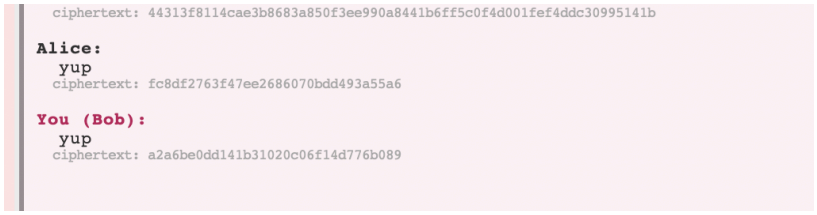
The GUI was built using tkinter which is a common python GUI. Each window shows the plaintext of sent and received messages with the ciphertext displayed underneath in gray below their specific messages. Info bar at the top was added to show the algorithm used and the rotation so we always know.



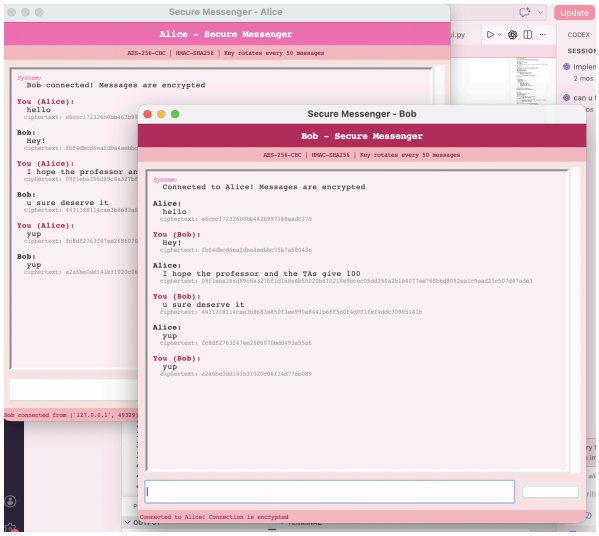
as long as you enter the same password for both alice and bob window



you can see which name for each window, the algorithm used, the rotation note. Looks like this when successfully connected. If not connected, retry and make sure you are entering the same password on both popups for both windows.



cipher text is displayed below the plaintext, you can see the exact same plaintext generates a different cipher text



just to admire the GUI design.

CONCLUSION

I wanted to do this project because it felt like a good opportunity to actually implement what we had been learning in class and get to see it all working through a real GUI at the end. Going through the lectures is helpful but actually implementing things makes more sense to me. I also learned a lot just from having to look things up on my own, most of the implementations came from going through cryptography.io and the Python docs and figuring out how they connected to what we covered. That helped me understand the material more than I expected it to and I linked it all inside the files.

If I were to keep working on this I would want to add Diffie-Hellman so Alice and Bob do not need a pre-shared password, and eventually get it running over an actual network instead of just localhost. Overall I am happy with how it came out!